lecture 7
Searching and binary search trees

think of a database of paper objects
16 the objects in the database

Operations
- insert object into database
- remove an object
- search. Is this object in the database

insert 5
insert 6
delete 5
search 6 ✓
search 5 ✗

example: compiler keeps track of used variables

Ideas
- store database entries in an array
- search through array
- append/remove ends in O(1) time
- swaps and comparisons between elements

dynamic search

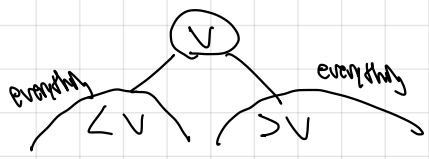
	insert	delete	search	
sorted array	O(n)	O(n)	O(log(n))	binary search
unsorted array	O(1)	O(n)	O(n)	

Sorted ⇝ binary search, but we need a better data structure for a better insertion and deletion runtime

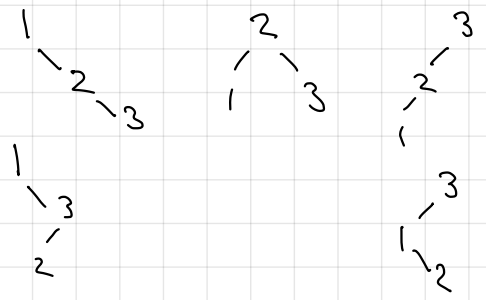
---

Binary Search Tree

Def: a binary tree where all nodes look like

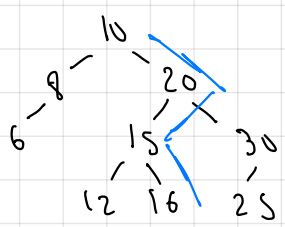


all BSTs of elements 1,2,3



BST encodes the sorted order!

In-order traversal outputs the sorted array
- visit left child
- visit node
- visit right child



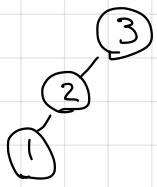
search 16
- traverse the tree. you know which side to go to next

search 17
- node not found

insert 17
- find correct spot and insert it if it isn't already there

```
def search_or_insert(root, v, do_insert):
    if root is empty:  # not found
        if not do_insert:
            return NOT_FOUND
        node = new node
        node.value = v
        replace root with node
        return node
    if v == root.value:
        return root
    if v < root.value
        return search_or_insert(root.left, v, do_insert)
    if v > root.value
        return search_or_insert(root.right, v, do_insert)
```

Insert 3, 2, 1



The worst case order is an array sorted in ascending or descending order, which returns in a linked list.

$\sum_{i=1}^{n}[i-1] = \frac{n(n+1)}{2} - (n-1+1)$

$= \frac{n(n+1)}{2} - \frac{2n}{2}$

$= \frac{n(n-1)}{2}$

each insertion is still O(n)

Minimum: left most

Maximum: right most

Deletion

- assume you found the node to delete

needs to maintain sorted order

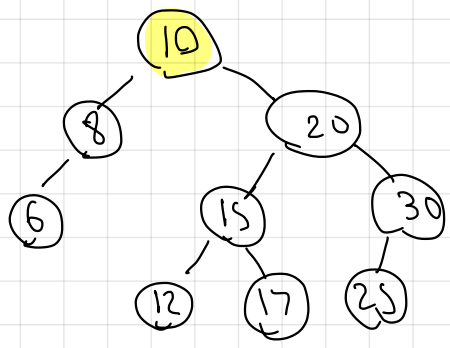
- If you have no children, delete is easy - just delete
- If you only have 1 child, the child is the new parent.
- If you have two children:
  - find the minimum on the right child
    - right child's left most element
  - swap successor with node
  - delete successor

```
def delete(node):
    if node.left is empty:
        replace node with node.right and return
    if node.right is empty:
        replace node with node.left and return
    successor = find_min(node.right)
    swap node.value <-> successor
    replace successor with successor.right and return
```

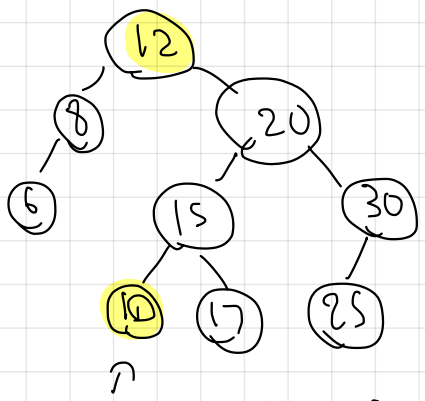
```
def find_min(root):
    if root is empty:
        error
    if root.left:
        return find_min(root.left)
    return root
```

* alternative: largest on left side
  smallest on right side

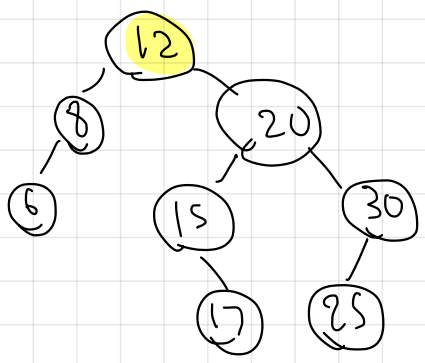
6, 8, 10, 12, 15, 17, 20, 25, 30



delete 10. successor is 12



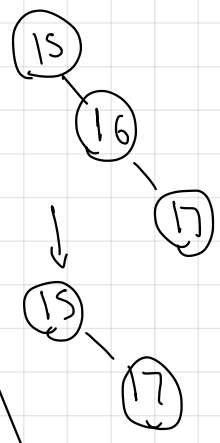
now, delete since successor does not have right node



* maintains in-order traversal order

6, 8, 12, 15, 17, 20, 25, 30

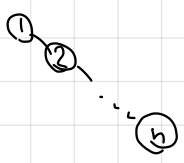
delete 16

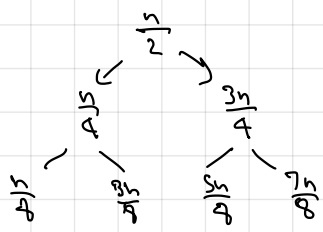


Time

Operation	Best Case (root)	Worst Case (linked list)
insert	O(1)	O(n)
search	O(1)	O(n)
delete	O(1)	O(n)
find-min	O(1)	O(n)
find-max	O(1)	O(n)

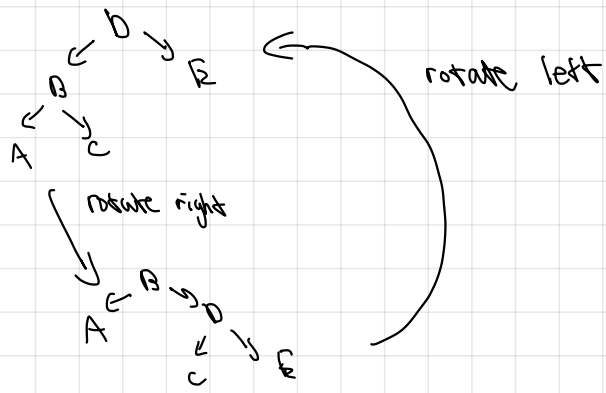
left or right location





Rotation

- manipulate tree structure while maintaining BST structure
- keep worst case operation low



recurrence:

average case 1 to n height of BST:

1. Permute $[1,\dots,n]$
2. Insert to BST
3. count comparisons

root: r — r-1 elements left of root, n-r elements right of root

$$T(n) = (n-1) + \frac{1}{n}\sum_{r=1}^{n}\left[T(r-1) + T(n-r)\right]$$

- $(n-1)$: all comparisons against the root
- $T(r-1)$: r-1 elements left of root
- $T(n-r)$: n-r element right of root
- $T(r-1) + T(n-r)$: call recurrence again

Solve recurrence — both are $T(0), T(1), \dots, T(n-1)$

$$T(n) = (n-1) + \frac{1}{n}\sum_{r=1}^{n}\left[T(r-1) + T(n-r)\right]$$

$$T(n) = (n-1) + \frac{2}{n}\sum_{i=0}^{n-1}\left[T(i)\right]$$

if constant, it's linear

generally, solve recurrences by looking at difference between terms $T(n) - T(n-1)$,
but we have different denominators, so hard to combine like terms

$$nT(n) - (n-1)T(n-1) = n\left[(n-1) + \frac{2}{n}\sum_{i=1}^{n-1}T(i)\right] - (n-1)\left[(n-2) + \frac{2}{n-1}\sum_{i=1}^{n-2}T(i)\right]$$
$$= n(n-1) + 2\sum_{r=1}^{n-1}T(i) - (n-1)(n-2) - 2\sum_{i=1}^{n-2}T(i)$$

subtract everything except $T(n-1)$

$$nT(n) - (n-1)T(n-1) = (n-1)(n-(n-2)) + 2T(n-1)$$
$$nT(n) - (n-1)T(n-1) = 2(n-1) + 2T(n-1)$$

$$nT(n) = 2n-2 + 2T(n-1) + (n-1)T(n-1)$$
$$nT(n) = 2n-2 + (n+1)T(n-1)$$
$$T(n) = 2 - \frac{2}{n} + \frac{(n+1)T(n-1)}{n}$$
$$\frac{T(n)}{n+1} = \frac{2}{n+1} - \frac{2}{n(n+1)} + \frac{T(n-1)}{n}$$

Define $S_n = \frac{T(n)}{(n+1)}$

$$S_n = \frac{2}{n+1} - \frac{2}{n(n+1)} + S_{n-1}$$ arithmetic sequence

$$\leq \frac{2}{n+1} + \frac{2}{(n-1)+1} + \frac{2}{(n-2)+1} + \dots + \frac{2}{1}$$
$$= O(\log(n))$$



$$S\frac{1}{x} = \log(x)$$

$$S_n = \frac{T(n)}{n+1}$$
$$T(n) = (n+1)S_n$$

$$\boxed{T(n) = O(n\log(n))}$$